

به نام خدا

پروژه کامپیوتری دوم درس مدار منطقی، استاد علیزاده

۸۱۰۱۰۲۴۶۷

محمدشهاب شرافت

برای این پروژه، ابتدا مدارهای Carry Lookahead Adder و Comparator چهار بیتی را روی کاغذ طراحی کردم و سپس آنها را روی سیستم وریلگ پیاده سازی کردم. سپس ماژول های Encoder و Decoder را ساختم و در نهایت مدار ALU را روی کاغذ کشیده و روی کد پیاده سازی کردم. در پایان یک Instance از ALU گرفتم و آن را داخل ماژول Top Module گذاشتم.

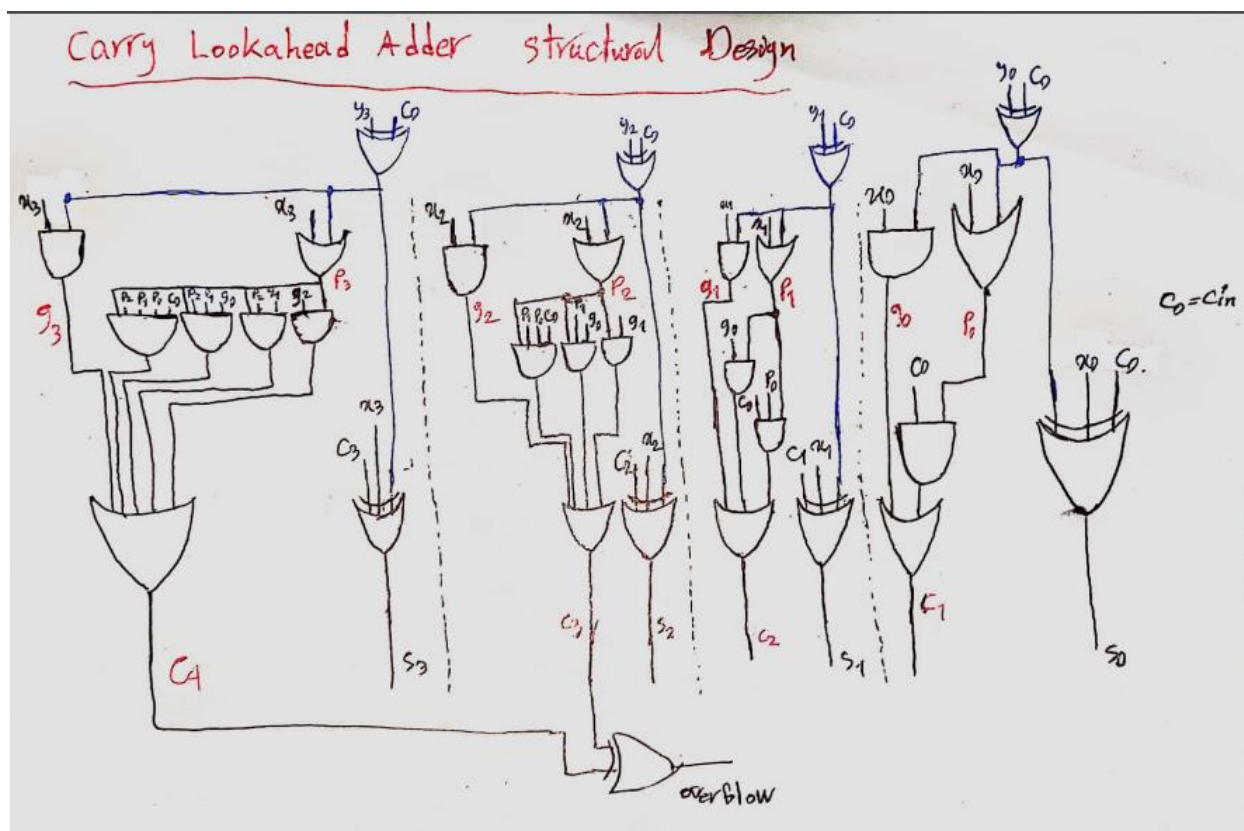
سپس فایل testbench.sv را نوشتم که شامل یک تست بنچ برای مقایسه گر، یکی برای Adder و یکی برای ALU و یکی برای تاپ ماژول است و شبیه سازی های لازم را با سه سری نمونه برای هرکدام از هفت عمل موردنیاز انجام دادم.

Carry Lookahead Adder(CLA)

برای ساختن این مدار، ابتدا ۴ تا بیت Carry 1 تا Carry 4 را با استفاده از p ها و g ها و C0 محاسبه کردم:

$$\begin{aligned}C_1 &= g_0 + p_0 C_0 = g_0 + (p_0 + g_0) C_0 \\&\Rightarrow C_{i+1} = g_i + p_i C_i \\&\Rightarrow C_2 = g_1 + p_1 (g_0 + p_0 C_0) = \\&\quad g_1 + p_1 g_0 + p_1 p_0 C_0 \\&\Rightarrow C_3 = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 C_0 \\&\Rightarrow C_4 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 C_0\end{aligned}$$

و سپس مدار این ادر چهاربیتی را در شکل زیر در سطح گیت طراحی کردم:

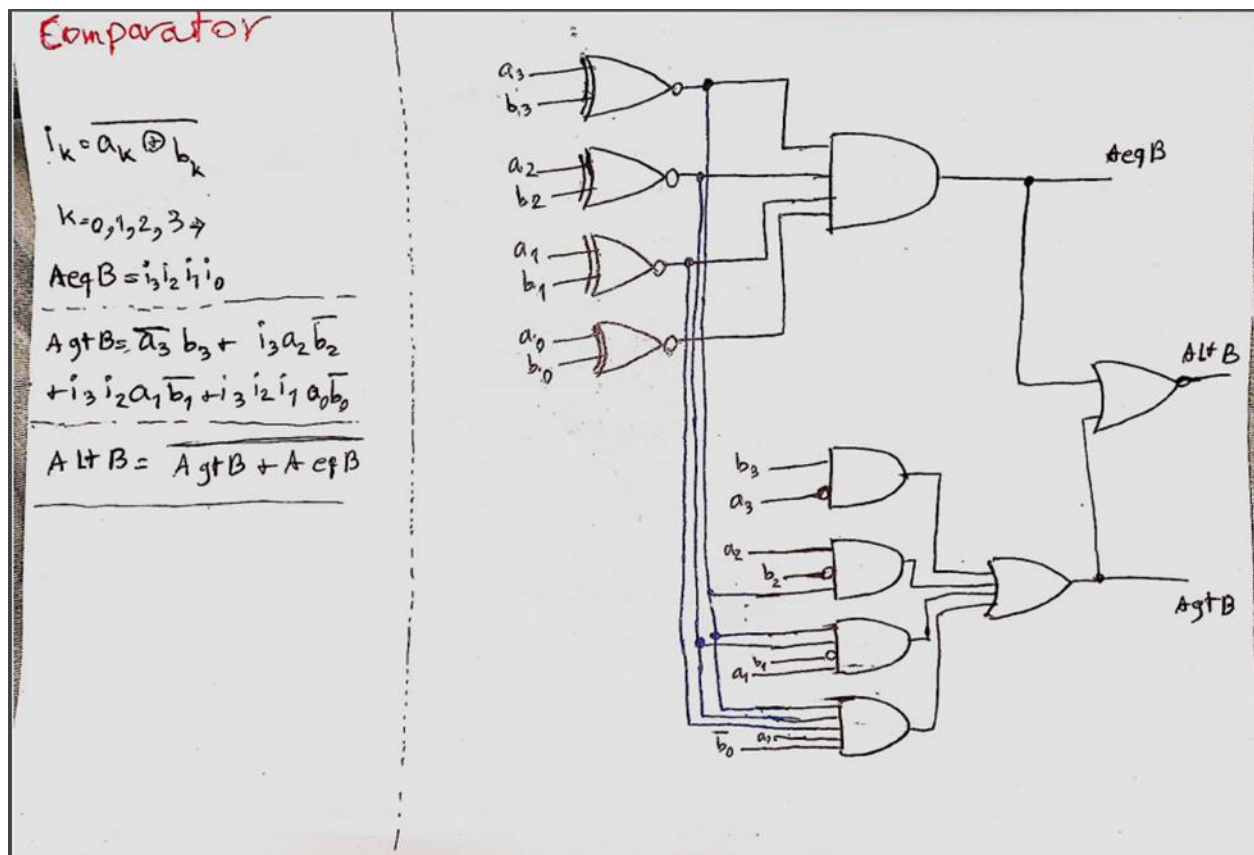


برای اینکه همزمان منها کردن را هم ساپورت کند، بیت های ورودی دوم مدار را یکی یکی با cin، xor کردم تا در صورت نیاز عدد 2's complement شود و با عدد اول جمع شود. همچنین برای تشخیص overflow، سومین و چهارمین بیت های نقلی را با هم xor کردم.

برای پیاده سازی کد این مدار هم از چند generate statement استفاده کردم و ساختمش.

Comparator

برای ساختن یک مقایسه گر چهاربیتی با علامت، ابتدا مقادیر i_0 تا i_3 (xor) های بیت های متناظر از دو عدد را محاسبه کردم و سپس تغییر ریزی در مقایسه گر ۴ بیتی موجود در اسلاید ها دادم. آن هم این بود که تنها تفاوتی که دارند این است که در MSB دو عدد که همان علامت آنهاست، ترتیب بزرگتر بودن فرق میکند. یعنی ۰ بزرگتر از ۱ است (چون مثبت ۰ است و منفی ۱).

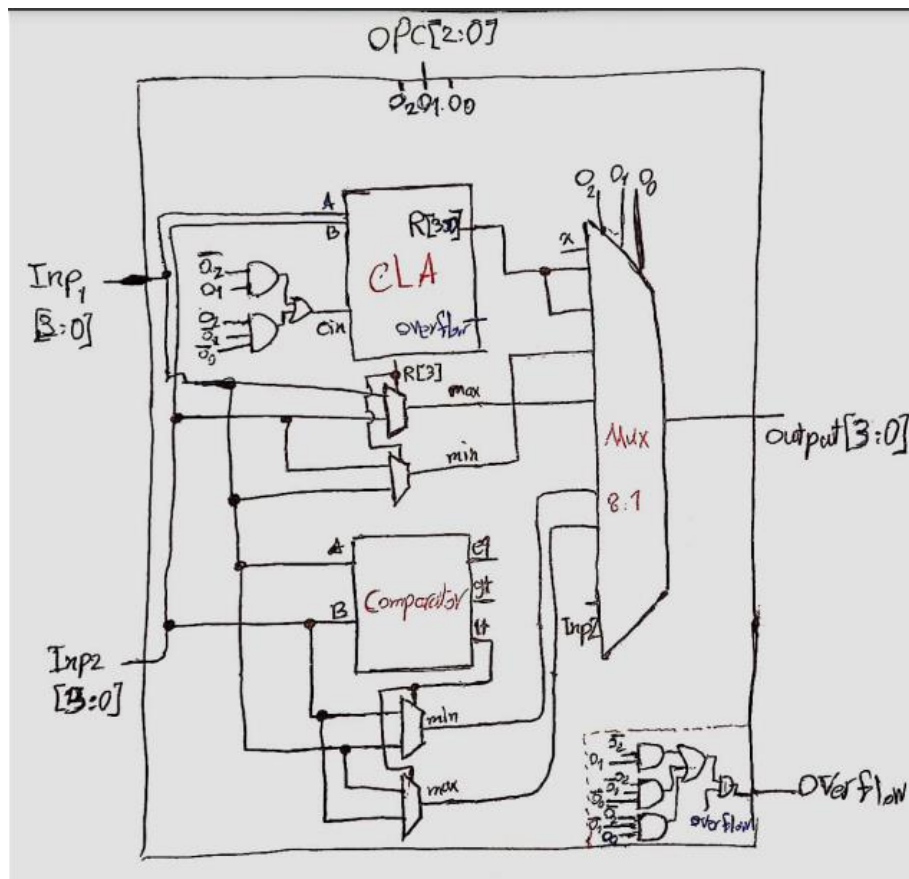


Arithmetic Logic Unit(ALU)

برای ساختن اصل کار یعنی ALU، ابتدا یک Instance از CLA گرفتیم. ورودی های آن را همان اعداد ۴ بیتی انکود شده دادم که قرار است Top Module تامینشان کند و همچنین Cin آن را با استفاده از گیت های منطقی و بهینه سازی طوری طراحی کردم که اگر opcode، حالت دوم، سوم یا چهارم بود یک باشد؛ چرا که در این صورت ها نیاز داریم دو عدد را از هم کم کنیم و Carry in باید ۱ باشد. سپس برای حالت های سوم و چهارم برای هرکدام یک مالتی پلکسر گذاشتم که با استفاده از آنها min و max را تعیین کنم (اگر بیت علامت ۱ باشد یعنی inp1 از inp2 کمتر بوده و مینیمم inp1 است).

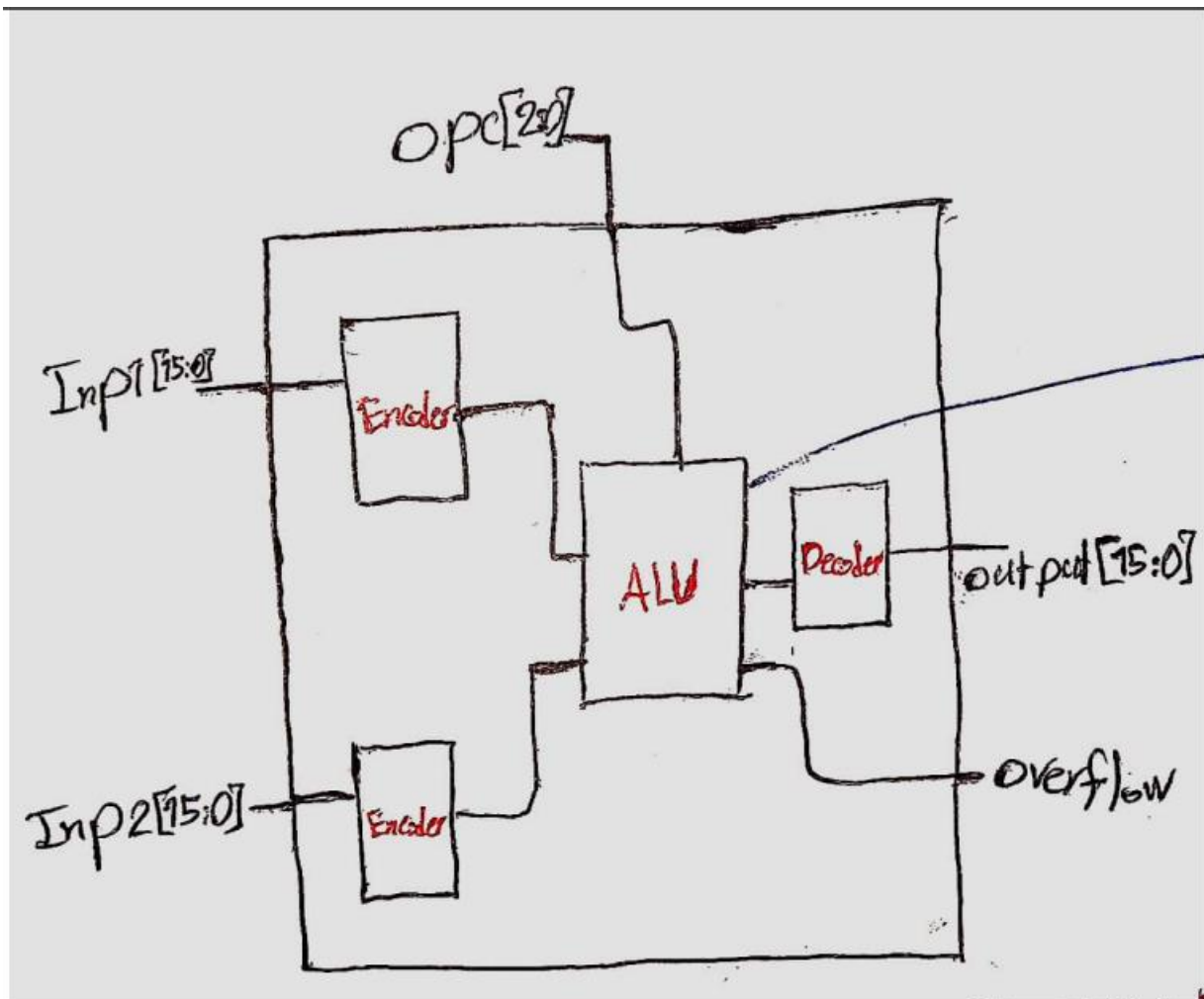
سپس یک اینستنس هم از Comparator ساختم و ورودی هایش را اعداد ۴ بیتی ورودی دادم و دوتا مالتی پلکسر هم برای min و max این حالت گذاشتم.

در نهایت همه ی خروجی های موردنیاز را به یک مالتی پلکسر بزرگ ۸ به ۱ میدهم تا یک ورودی را انتخاب و به خروجی بفرستد.



Top Module

برای ساختن تاپ ماژول هم دوتا انکودر برای ورودی ها گذاشتم و تبدیل به ۴ بیتی کردمشان. سپس یک اینستنس از ALU ساختم و ورودی های ۴ بیتی و opcode را به آن دادم تا عملیات لازم را روی آنها انجام بدهد. در نهایت خروجی ۴ بیتی ALU را گرفته، دیکود کرده و خروجی نهایی ۱۶ بیتی را به همراه خروجی overflow روی خروجی نهایی Top Module میفرستم.



TestBench

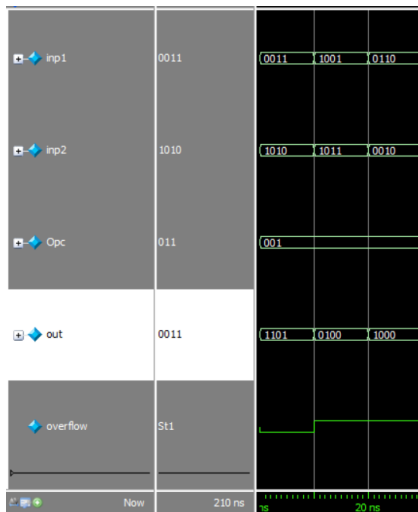
سه سری تست کیس طراحی کردم و در فایل تست بنچ نوشتم و روی هر ۷ حالت opcode امتحانشان کردم.

سری اول: $\text{inp1} = +3, \text{inp2} = -6$

سری دوم: $\text{inp1} = -7, \text{inp2} = -5$

سری سوم: $\text{inp1} = 6, \text{inp2} = 2$

حالا یکی یکی برای هر هفت حالت عکس شبیه سازی را میگذارم و تحلیل میکنم.

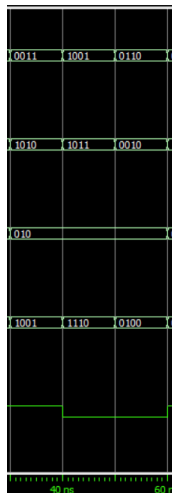


Opcode = 001

سری اول: $\text{inp1} + \text{inp2} = -3$ و $\text{overflow} = 0$

سری دوم: $\text{inp1} + \text{inp2} = -12$ و $\text{overflow} = 1$

سری سوم: $\text{inp1} + \text{inp2} = 8$ و $\text{overflow} = 1$

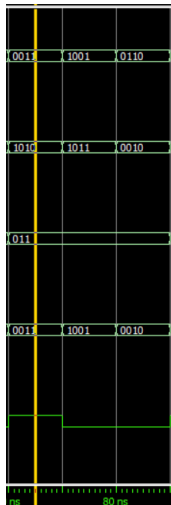


Opcode = 010

سری اول: $\text{inp1} - \text{inp2} = 9$ و $\text{overflow} = 1$

سری دوم: $\text{inp1} - \text{inp2} = -2$ و $\text{overflow} = 0$

سری سوم: $\text{inp1} - \text{inp2} = 4$ و $\text{overflow} = 0$

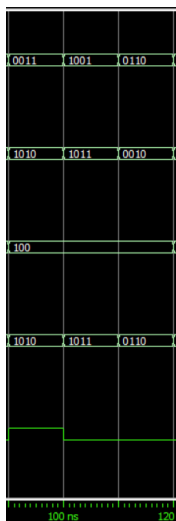


Opcode = 011

سری اول: $\min(\text{inp1}, \text{inp2})$: Invalid! و $\text{overflow} = 1$

سری دوم: inp1 : $\min(\text{inp1}, \text{inp2})$ و $\text{overflow} = 0$

سری سوم: inp2 : $\min(\text{inp1}, \text{inp2})$ و $\text{overflow} = 0$

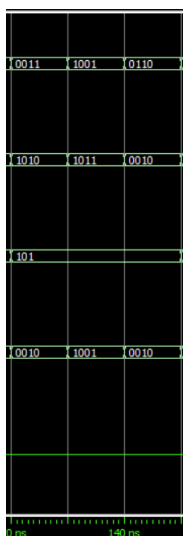


Opcode = 100

سری اول: $\max(\text{inp1}, \text{inp2})$: Invalid! و $\text{overflow} = 1$

سری دوم: inp2 : $\max(\text{inp1}, \text{inp2})$ و $\text{overflow} = 0$

سری سوم: inp1 : $\max(\text{inp1}, \text{inp2})$ و $\text{overflow} = 0$

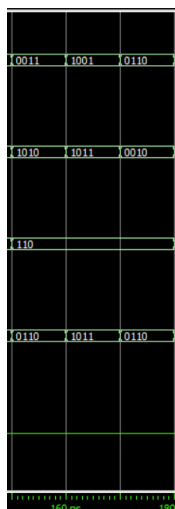


Opcode = 101

سری اول: $\min(\text{inp1}, \text{inp2})$: inp2 و $\text{lt} = 0$ و $\text{overflow} = 0$

سری دوم: inp1 : $\min(\text{inp1}, \text{inp2})$ و $\text{lt} = 1$ و $\text{overflow} = 0$

سری سوم: inp2 : $\min(\text{inp1}, \text{inp2})$ و $\text{lt} = 0$ و $\text{overflow} = 0$

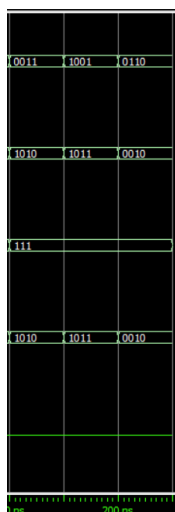


Opcode = 110

سری اول: inp1 : $\max(\text{inp1}, \text{inp2})$ و $\text{lt} = 0$ و $\text{overflow} = 0$

سری دوم: inp2 : $\max(\text{inp1}, \text{inp2})$ و $\text{lt} = 1$ و $\text{overflow} = 0$

سری سوم: inp1 : $\max(\text{inp1}, \text{inp2})$ و $\text{lt} = 0$ و $\text{overflow} = 0$

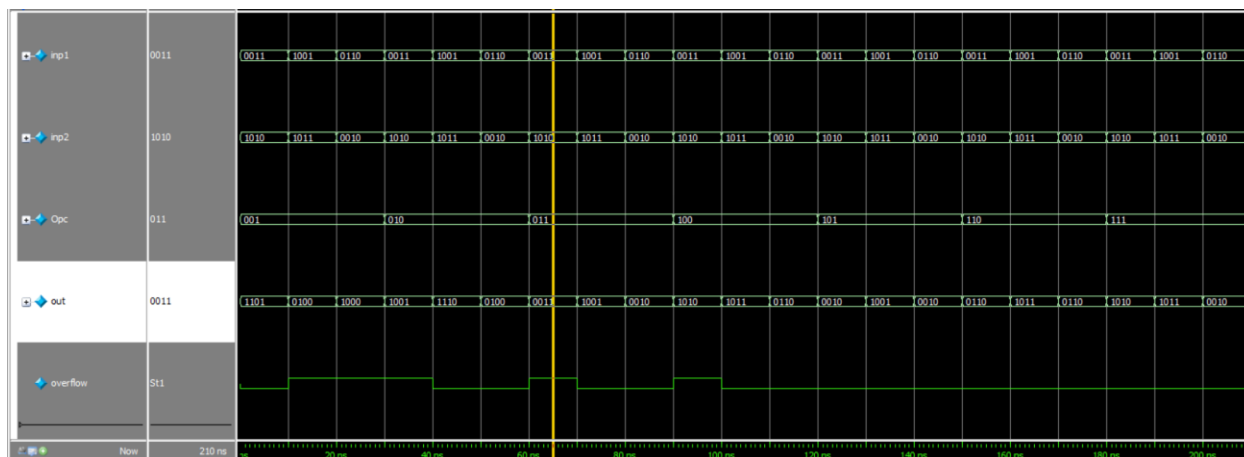


Opcode = 111

سری اول: inp2 : $\text{result} = \text{inp2}$ و $\text{overflow} = 0$

سری دوم: inp2 : $\text{result} = \text{inp2}$ و $\text{overflow} = 0$

سری سوم: inp2 : $\text{result} = \text{inp2}$ و $\text{overflow} = 0$



برای بخش امتیازی هم از پروژه ی اولم یک instance از ماژول seven segment decoder تغییر یافته ساختم. طوری تغییرش دادم که یک خروجی sign و یک خروجی magnitude بدهد که هرکدام بیانگر یک seven segment هستند.